

Dockerizing a Python Calculator Application

- 1 Requirements
 - 2 Application code
 - 3 Dockerfile
 - 4 Build steps
 - 5 Run steps
 - 6 Workflow explanation
-

☐ **PROJECT: Python Web Calculator (Flask Based)**

- Simple Web Calculator
 - Addition, Subtraction, Multiplication, Division
 - Run inside Docker container
-

✓ ☒ **1 REQUIREMENTS**

☐ **System Requirements**

- Docker installed (docker --version)
 - Python not required on host (Docker handles it)
-

☐ **Application Requirements**

We need:

- Python 3
- Flask library

Flask is a lightweight web framework.

☐ **2 PROJECT STRUCTURE**

Create folder:

```
mkdir pycalculator  
cd pycalculator
```

☐ **3 Create Python Application**

Create file:

```
nano app.py
```

Paste:

```

from flask import Flask, request, render_template_string

app = Flask(__name__)

html = """
<!DOCTYPE html>
<html>
<head>
    <title>Python Calculator</title>
</head>
<body>
    <h2>Simple Calculator</h2>
    <form method="POST">
        Number 1: <input type="number" name="num1" required><br><br>
        Number 2: <input type="number" name="num2" required><br><br>
        Operation:
        <select name="operation">
            <option value="add">Add</option>
            <option value="sub">Subtract</option>
            <option value="mul">Multiply</option>
            <option value="div">Divide</option>
        </select><br><br>
        <input type="submit" value="Calculate">
    </form>
    <h3>Result: {{ result }}</h3>
</body>
</html>
"""

@app.route("/", methods=["GET", "POST"])
def calculator():
    result = ""
    if request.method == "POST":
        num1 = float(request.form["num1"])
        num2 = float(request.form["num2"])
        operation = request.form["operation"]

        if operation == "add":
            result = num1 + num2
        elif operation == "sub":
            result = num1 - num2
        elif operation == "mul":
            result = num1 * num2
        elif operation == "div":
            result = num1 / num2 if num2 != 0 else "Cannot divide by zero"

    return render_template_string(html, result=result)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)

```

Save and exit.

☐ 4 Create requirements.txt

nano requirements.txt

Add:

flask

Save.

☐ 5 Create Dockerfile

nano Dockerfile

Paste:

FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

EXPOSE 5000

CMD ["python", "app.py"]

Save.

☐ Dockerfile Explanation

Instruction	Meaning
-------------	---------

FROM python	Base image
----------------	------------

WORKDIR	Working directory inside container
---------	------------------------------------

COPY	Copy files
------	------------

RUN	Install dependencies
-----	----------------------

EXPOSE 5000	App runs on port 5000
----------------	-----------------------

CMD	Start application
-----	-------------------

❑ 6 Build Docker Image

Inside project folder:

```
docker build -t pycalculator:v1 .
```

Check image:

```
docker images
```

▶ 7 Run Container

```
docker run -d -p 8080:5000 --name pycalccontainer pycalculator:v1
```

Explanation:

Host Port 8080 → Container Port 5000 <http://localhost:8080>

❑ 8 Open Browser

You now have a working Dockerized Python Calculator ❑

❑ Debug Commands (Lab)

Check running container:

```
docker ps
```

View logs:

```
docker logs pycalccontainer
```

Stop:

```
docker stop pycalccontainer
```

Remove:

```
docker rm pycalccontainer
```

❑ Internal Architecture

Browser



Port 8080 (Host)



Docker Engine



Container Port 5000



Flask Application

↓
Python Logic

□ **Teaching Points for Students**

Important concepts covered:

- Dockerizing backend app
- Dependency management
- Port mapping
- Image vs Container
- Layered architecture